

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Колледж программирования и кибербезопасности

УЧЕБНАЯ ДИСЦИПЛИНА/
МЕЖДИСЦИПЛИНАРНЫЙ КУРС
МДК. 02.01 Технология разработки программного обеспечения

ПЦК ***Программирование***

ВИД УЧЕБНОГО
МАТЕРИАЛА ***Лекция - 16***

ПРЕПОДАВАТЕЛЬ ***Понеделко Елена Викторовна***

Лекция – 16

Тестовое покрытие. Тестовый сценарий, тестовый пакет

Анализ спецификаций. Верификация и аттестация программного обеспечения

Основной всплеск интереса к теме тестирования пришёлся на 90-е годы и начался в США. Бурное развитие систем автоматизированной разработки ПО (CASE-средств) и сетевых технологий привело к росту рынка производства ПО и к пересмотру вопросов обеспечения качества и надёжности разрабатываемых программ. Резко усилившаяся конкуренция между производителями ПО потребовала особого внимания к качеству создаваемых продуктов, т.к. теперь у потребителя был выбор: многие фирмы предлагали свои продукты и услуги по достаточно приемлемым ценам, а потому можно было обратиться к тем, кто разработает программу не только быстро и дёшево, но и качественно.

Ситуация осложнилась тем фактом, что компьютеризации подвержены практически все области человеческой жизни. И вопрос о качестве ПО начинает приобретать особую важность: сегодня это уже не только комфорт от работы в той или иной программе, сегодня ПО управляет оборудованием в больницах, диспетчерскими системами в аэропортах, атомными реакторами, космическими кораблями и т.д.

Осознав тот факт, что обеспечение высокого качества разрабатываемого ПО – это реальный путь «обойти» конкурентов, многие компании во всём мире вкладывают всё больше средств в обеспечение качества своих продуктов, создавая собственные группы и отделы, занимающиеся тестированием, или передавая тестирование своих продуктов сторонним организациям.

Наиболее крупные компании, заботящиеся о своей репутации и желающие пройти сертификацию на высокий уровень CMMI (Capability Maturity Model Integration) создают свои собственные системы управления качеством (Quality Management System), направленные на постоянное совершенствование производственных процессов и постоянное повышение качества ПО.

Сегодня тестирование стало обязательной частью процесса производства ПО. Оно направлено на обнаружение и устранение как можно большего числа ошибок. Следствием такой деятельности является повышение качества ПО по всем его характеристикам.

Существующие на сегодняшний день методы тестирования программного обеспечения не позволяют однозначно и полностью устранить все дефекты и ошибки и установить корректность функционирования программного продукта. Поэтому все существующие методы тестирования действуют в рамках формального процесса проверки исследуемого или разрабатываемого программного продукта. Такой процесс формальной проверки или верификации может доказать, что дефекты отсутствуют с точки зрения используемого метода. То есть нет никакой возможности точно установить или гарантировать отсутствие дефектов в программном продукте с учётом человеческого фактора, присутствующего на всех этапах жизненного цикла программного обеспечения.

Существует множество подходов к решению задачи тестирования и верификации программного обеспечения, но эффективное тестирование сложных программных продуктов — это процесс в высшей степени творческий, не сводящийся к следованию строгим и чётким процедурам или созданию таковых. Конечной целью любого процесса тестирования является обеспечение такого ёмкого (совокупного) понятия как Качество с учётом всех или наиболее критичных для данного конкретного случая составляющих.

Тестирование программного обеспечения — попытка определить, выполняет ли программа то, что от неё ожидают. Как правило, никакое тестирование не может дать абсолютной гарантии работоспособности программы в будущем. Задача тестирования

программного обеспечения – снизить стоимость разработки путем раннего обнаружения дефектов.

ОПРЕДЕЛЕНИЕ ТЕСТИРОВАНИЯ

В соответствии с IEEE Std 829-1983 Тестирование — это процесс анализа ПО, направленный на выявление отличий между его реально существующими и требуемыми свойствами (дефект) и на оценку свойств ПО.

По ГОСТ Р ИСО МЭК 12207-99 в жизненном цикле ПО определены среди прочих вспомогательные процессы верификации, аттестации, совместного анализа и аудита.

Процесс верификации является процессом определения того, что программные продукты функционируют в полном соответствии с требованиями или условиями, реализованными в предшествующих работах. Данный процесс может включать анализ, проверку и испытание (тестирование).

Процесс аттестации является процессом определения полноты соответствия установленных требований, созданной системы или программного продукта их функциональному назначению.

Процесс совместного анализа является процессом оценки состояний и, при необходимости, результатов работ (продуктов) по проекту.

Процесс аудита является процессом определения соответствия требованиям, планам и условиям договора.

В сумме эти процессы и составляют то, что обычно называют тестированием.

Тестирование основывается на тестовых процедурах с конкретными входными данными, начальными условиями и ожидаемым результатом, разработанными для определенной цели, такой, как проверка отдельной программы или верификация соответствия на определенное требование. Тестовые процедуры могут проверять различные аспекты функционирования программы — от правильной работы отдельной функции до адекватного выполнения бизнес-требований.

При выполнении проекта необходимо учитывать, в соответствии с какими стандартами и требованиями будет проводиться тестирование продукта, какие инструментальные средства будут (если будут) использоваться для поиска и для документирования найденных дефектов. Если помнить о тестировании с самого начала выполнения проекта, тестирование разрабатываемого продукта не доставит неприятных неожиданностей, а значит и качество продукта, скорее всего, будет достаточно высоким.

МЕТОДОЛОГИЯ ТЕСТИРОВАНИЯ

В терминологии профессионалов тестирования (программного и некоторого аппаратного обеспечения), фразы «тестирование белого ящика» и «тестирование чёрного ящика» относятся к тому, имеет ли разработчик тестов доступ к исходному коду тестируемого программного обеспечения, или же тестирование выполняется через пользовательский интерфейс либо прикладной программный интерфейс, предоставленный тестируемым модулем.

При тестировании методом **белого ящика** (white-box testing, также говорят — прозрачного ящика, оно же структурное тестирование), разработчик теста имеет доступ к исходному коду программ и может писать код, который связан с библиотеками тестируемого программного обеспечения.

Тесты создаются на основе знаний о структуре самой системы и о том, как она работает. Критерии полноты основаны на проценте элементов кода, которые отработали в ходе выполнения тестов. Для оценки степени соответствия требованиям могут привлекаться дополнительные знания о связи требований с определенными ограничениями на значения внутренних данных системы (например, на значения параметров вызовов, результатов и локальных переменных).

При тестировании методом **чёрного ящика** тестировщик имеет доступ к программному обеспечению только через те же интерфейсы, что и заказчик или пользователь, либо через внешние интерфейсы, позволяющие другому компьютеру либо другому процессу подключиться к системе для тестирования.

Тестирование черного ящика нацелено на проверку требований. Тесты для него и критерий полноты тестирования строятся на основе требований и ограничений, четко зафиксированных в спецификациях, стандартах, внутренних нормативных документах. Часто такое тестирование называется тестированием на соответствие (conformance testing). Частным случаем его является функциональное тестирование — тесты для него, а также используемые критерии полноты проведенного тестирования определяют на основе требований к функциональности.

Помимо методов тестирования «белый ящик» и «черный ящик» различают тестирование методом «**серого ящика**». В данном случае у человека, который разрабатывает тест кейсы, есть некоторая информация о внутренней структуре приложения или о деталях реализации. Данный метод применяется в последнее время чаще предыдущих.

ТЕХНИКИ СОЗДАНИЯ ТЕСТОВ ДЛЯ ЧЕРНОГО ЯЩИКА

Целью тестирования ставится выяснение обстоятельств, в которых поведение программы не соответствует спецификации. Для обнаружения всех ошибок в программе необходимо выполнить исчерпывающее тестирование, то есть тестирование на всевозможных наборах данных. Для большинства программ такое невозможно, поэтому применяют разумное тестирование, при котором тестирование программы ограничивается небольшим подмножеством всевозможных наборов данных. При этом необходимо выбирать наиболее подходящие подмножества, подмножества с наивысшей вероятностью обнаружения ошибок. Свойства правильно выбранного теста:

– уменьшает более, чем на одно число других тестов, которые должны быть разработаны для разумного тестирования;

– покрывает значительную часть других возможных тестов, что в некоторой степени свидетельствует о наличии или отсутствии ошибки до и после ограниченного множества тестов.

Техники черного ящика:

– эквивалентное разбиение;
– анализ граничных значений;
– анализ причинно-следственных связей;
– попарное тестирование;
– предположение об ошибке.

Эквивалентное разбиение

Основу метода эквивалентное разбиение составляют два положения:

– исходные данные необходимо разбить на конечное число классов эквивалентности. В одном классе эквивалентности содержатся такие тесты, что, если один тест из класса эквивалентности обнаруживает некоторую ошибку, то и любой другой тест из этого класса эквивалентности должен обнаруживать эту же ошибку;

– каждый тест должен включать, по возможности, максимальное количество классов эквивалентности, чтобы минимизировать общее число тестов.

Разработка тестов этим методом осуществляется в два этапа: выделение классов эквивалентности и построение теста. Эквивалентный класс — это одно или больше значений ввода, к которым ПО применяет одинаковую логику.

Классы эквивалентности выделяются путём выбора каждого входного условия, которые берутся с помощью технического задания или спецификации и разбиваются на две и более группы. Выделение классов эквивалентности является эвристическим способом, однако существует ряд правил:

– если входное условие описывает область значений, например «Целое число принимает значение от 0 до 999», то существует один правильный класс эквивалентности и два неправильных;

– если входное условие описывает число значений, например «Число строк во входном файле лежит в интервале (1..6)», то также существует один правильный класс и два неправильных;

– если входное условие описывает множество входных значений, то определяется количество правильных классов, равное количеству элементов в множестве входных значений. Если входное условие описывает ситуацию «должно быть», например «Первый символ должен быть заглавным», тогда один класс правильный и один неправильный;

– если есть основание считать, что элементы внутри одного класса

эквивалентности могут программой трактоваться по-разному, необходимо разбить данный класс на подклассы. На этом шаге тестирующий на основе таблицы должен составить тесты, покрывающие собой все правильные и неправильные классы эквивалентности. При этом составитель должен минимизировать общее число тестов.

Несколько тестов эквивалентны, если:

– они направлены на поиск одной и той же ошибки;

– если один из тестов обнаруживает ошибку, другие ее тоже, скорей всего, обнаружат;

– если один из тестов не обнаруживает ошибку, другие ее тоже, скорей всего, не обнаружат;

– тесты используют один и те же наборы входных данных;

– для выполнения тестов приходится совершать одни и те же операции;

– тесты генерируют одинаковые выходные данные или приводят приложение в одно и то же состояние;

– все тесты приводят к срабатыванию одного и того же блока обработки ошибок;

– ни один из тестов не приводит к срабатыванию блока обработки ошибок.

Определение тестов:

– каждому классу эквивалентности присваивается уникальный номер;

– если еще остались не включенные в тесты правильные классы, то пишутся тесты, которые покрывают максимально возможное количество классов;

– если остались не включенные в тесты неправильные классы, то пишут тесты, которые покрывают только один класс.

Анализ граничных значений

Граничные условия — это ситуации, возникающие на высших и нижних границах входных классов эквивалентности.

Анализ граничных значений отличается от эквивалентного разбиения следующим:

– выбор любого элемента в классе эквивалентности в качестве представительного осуществляется таким образом, чтобы проверить тестом каждую границу этого класса;

– при разработке тестов рассматриваются не только входные значения (пространство входов), но и выходные (пространство выходов). Метод требует определённой степени творчества и специализации в рассматриваемой задаче.

Существует несколько правил:

1) построить тесты с неправильными входными данными для ситуации незначительного выхода за границы области значений. Если входные значения должны быть в интервале [-1.0 .. +1.0], нужно проверить −1.0, 1.0, −1.000001, 1.000001;

2) обязательно писать тесты для минимальной и максимальной границы диапазона;

3) если вход и выход программы представляет упорядоченное множество, сосредоточить внимание на первом и последнем элементах списка.

Анализ граничных значений, если он применён правильно, позволяет обнаружить большое число ошибок. Однако определение этих границ для каждой задачи может являться отдельной трудной задачей. Также этот метод не проверяет комбинации входных значений.

Анализ причинно-следственных связей

Анализ причинно-следственных связей позволяет системно выбирать высокорезультативные тесты. Метод использует алгебру логики и оперирует понятиями «причина» и «следствие». Причиной в данном случае называют отдельное входное условие или класс эквивалентности. Следствием - выходное условие или преобразование системы.

Идея метода заключается в отнесении всех следствий к причинам, т. е. в уточнении причинно-следственных связей. Данный метод дает полезный побочный эффект, позволяя обнаруживать неполноту и неоднозначность исходных спецификаций.

Построение тестов осуществляют в несколько этапов. Сначала, поскольку таблицы причинно-следственных связей при применении метода к большим спецификациям становятся громоздкими, спецификации разбивают на «рабочие» участки, стараясь по возможности выделять в отдельные таблицы независимые группы причинно-следственных связей. Затем в спецификации определяют множество причин и следствий.

Далее на основе анализа семантического (смыслового) содержания спецификации строят таблицу истинности, в которой каждой возможной комбинации причин ставится в соответствие следствие. При этом целесообразно истину обозначать «I», ложь - «О», а для обозначения безразличных состояний условий применять обозначение «Х», которое предполагает произвольное значение условия (0 или 1). Таблицу сопровождают примечаниями, задающими ограничения и описывающими комбинации причин и/или следствий. которые являются невозможными из-за синтаксических или внешних ограничений. При необходимости аналогично строится таблица истинности для класса эквивалентности. И, наконец, каждую строку таблицы преобразуют в тест. При этом рекомендуется по возможности совмещать тесты из независимых таблиц. Данный метод позволяет строить высокорезультативные тесты и обнаруживать неполноту и неоднозначность исходных.

Попарное тестирование

Pairwise testing - это техника формирования наборов тестовых данных, в которых каждое тестируемое значения каждого из проверяемых параметров хотя бы единожды сочетается с каждым тестируемым значением всех остальных проверяемых параметров. Для демонстрации данной техники берется в качестве примера абстрактная система с большим числом параметров, влияющих на её работу, которую нужно протестировать.

Опытный тестировщик знает, что для проверки всех комбинации не хватит времени. К примеру, для проверки всех сочетаний 10 параметров с 10 значениями каждый, потребуется 10 000 000 000 тестов, в то время как метод перебора пар позволяет реализовать сравнимое по качеству тестирование (учитывая количество и критичность найденных в результате багов) используя всего 177 тестов. Метод парного тестирования основан на довольно простой, но от того не менее эффективной идее, что подавляющее большинство ошибок выявляется тестом, проверяющим один параметр, либо сочетание двух. Ошибки, причиной которых явились комбинации трех и более параметров как

правило значительно менее критичны, чем пары параметров или одного. Если этого требует ситуация, то можно дополнить тестовое покрытие кейсами на желаемые комбинации параметров. Перебрать все пары не сложно, трудность заключается в том, чтобы обеспечить при этом минимум тестов, комбинируя проверки нескольких пар в одном тесте. Математические методы могут обеспечить такой необходимый минимум тестов. Одним из таких методов являются ортогональные матрицы.

Предположение об ошибке

Программист с большим опытом выискивает ошибки без всяких методов, но при этом он подсознательно использует метод предположения об ошибке. Данный метод в значительной степени основан на интуиции. Основная идея метода состоит в том, чтобы составить список, который перечисляет возможные ошибки и ситуации, в которых эти ошибки могли проявиться. Потом на основе списка составляются тесты.

УРОВНИ ТЕСТИРОВАНИЯ

Существует классификация видов тестирования, которая основана на том уровне детализации работ проекта, на который оно нацелено. Разновидности тестирования можно связать с фазой жизненного цикла, на которой они выполняются.

Модульное тестирование (Unit-testing) — уровень тестирования, на котором тестируется минимально возможный для тестирования компонент, например, отдельный класс или функция. На этом уровне применяются методы «белого ящика». В современных проектах модульное тестирование («юнит-тестинг») осуществляется разработчиками.

Модульное тестирование предназначено для проверки правильности отдельных модулей, вне зависимости от их окружения. При этом проверяется, что если модуль получает на вход данные, удовлетворяющие определенным критериям корректности, то и результаты его корректны. Для описания критериев корректности входных и выходных данных часто используют программные контракты — предусловия, описывающие для каждой операции на каких входных данных она предназначена работать, постусловия, описывающие для каждой операции, как должны соотноситься входные данные с возвращаемыми ею результатами, и инварианты, определяющие критерии целостности внутренних данных модуля.

Основной недостаток модульного тестирования состоит в том, что проводить его можно, только когда проверяемый элемент программы уже разработан. Снизить влияние этого ограничения можно, подготавливая тесты (а это — наиболее трудоемкая часть тестирования) на основе требований заранее, когда исходного кода еще нет.

Модульное тестирование является важной составной частью отладочного тестирования, выполняемого разработчиками для отладки написанного ими кода.

Интеграционное тестирование (Integration testing) – уровень тестирования, на котором отдельные программные модули объединяются и тестируются в группе. Обычно интеграционное тестирование проводится после модульного тестирования (юнит-тесты для модулей должны быть выполнены и найденные ошибки исправлены).

Интеграционное тестирование в качестве входных данных использует модули, над которыми было проведено модульное тестирование, группирует их в более крупные множества, выполняет тесты, определённые в плане тестирования для этих множеств, и представляет их в качестве выходных данных и входных для последующего системного тестирования. При этом проверяется, что в ходе совместной работы модули обмениваются данными и вызовами операций, не нарушая взаимных ограничений на такое взаимодействие, например, предусловий вызываемых операций.

Интеграционное тестирование выполняется разработчиками, используется при отладке, но на более позднем этапе разработки.

Системное тестирование (System testing) - это тестирование программного обеспечения, выполняемое на полной, интегрированной системе с целью проверки соответствия системы исходным требованиям.

Системное тестирование относится к методам тестирования «чёрного ящика», и, тем самым, не требует знаний о внутреннем устройстве системы. Системное тестирование выполняется через внешние интерфейсы программного обеспечения и тесно связано с тестированием пользовательского интерфейса (или через пользовательский интерфейс), проводимым при помощи имитации действий пользователей над элементами этого интерфейса. Частными случаями этого вида тестирования являются тестирование графического пользовательского интерфейса (Graphical User Interface, GUI) и пользовательского интерфейса Web-приложений (WebUI). Системное тестирование выполняется инженерами по тестированию.

Приемочное тестирование (Acceptance testing) - это тестирование готового продукта конечными пользователями на реальном окружении, в котором будет функционировать тестируемое приложение. Приемочные тесты разрабатываются пользователями, обычно, в виде сценариев. Для того, чтобы найти больше ошибок, рекомендуется планировать не только системное тестирование и приемочное, но и модульное, и интеграционное.

Статическое тестирование (Static testing) – тестирование, в ходе которого тестируемая программа (код) не выполняется (запускается). Анализ программы происходит на основе исходного кода, который вычитывается вручную, либо анализируется специальными инструментами.

Примеры статического тестирования:

– обзоры (Reviews);
– инспекции (Inspections);
– сквозные просмотры (Walkthroughs);
– аудиты (Audits).

Динамическое тестирование (Dynamic testing) – тестирование, в ходе которого тестируемая программа (код) выполняется (запускается).

Альфа-тестирование — тестирование в процессе разработки. **Бета-тестирование** — тестирование выполняется пользователями (end-users).

Перед тем, как выпускается программное обеспечение, как минимум, оно должно проходить стадии альфа- (внутреннее пробное использование) и бета- (пробное использование с привлечением отобранных внешних пользователей) версий.

Отчеты об ошибках, поступающие от пользователей этих версий продукта, обрабатываются в соответствии с определенными процедурами, включающими подтверждающие тесты (любого уровня), проводимые специалистами группы разработки. Данный вид тестирования не может быть заранее спланирован.

Регрессионное тестирование (Regression testing) – тестирование функциональности, которая была уже протестирована до внесения какого-либо изменения.

После внесения изменений в очередную версию программы, регрессионные тесты подтверждают, что сделанные изменения не повлияли на работоспособность остальной функциональности приложения. Регрессионное тестирование может выполняться как вручную, так и средствами автоматизации тестирования.

Определение успешности регрессионных тестов (IEEE 610-90 “Standard Glossary of Software Engineering Terminology”) гласит: “повторное выборочное тестирование системы или компонент для проверки сделанных модификаций не должно приводить к непредусмотренным эффектам”. На практике это означает, что если система успешно проходила тесты до внесения модификаций, она должна их проходить и после внесения таковых.

Основная проблема регрессионного тестирования заключается в поиске компромисса между имеющимися ресурсами и необходимостью проведения таких тестов по мере внесения каждого изменения. В определенной степени задача состоит в том, чтобы определить критерии “масштабов” изменений, с достижением которых необходимо проводить регрессионные тесты.

«Смок-тест» (Smoke Testing, «дымовое тестирование») в тестировании означает минимальный набор тестов на явные ошибки. Дымовой тест обычно выполняется самим программистом; не проходящую этот тест программу не имеет смысла отдавать на более глубокое тестирование.

ВИДЫ ТЕСТИРОВАНИЯ (№2)

Функциональное тестирование (Functional testing) - цель данного тестирования состоит в том, чтобы убедиться в надлежащем функционировании объекта тестирования:

– каждое функциональное требование транслируется в тест-кейсы (используя техники «черного ящика») для того, чтобы проверить, что система функционирует в точности, как и описано в спецификации (функциональных требованиях к системе);

– проверятся, все ли функциональные требования действительно запрограммированы/реализованы.

Тестирование производительности (Perfomance testing) - тестирование, которое проводится с целью определения, как быстро работает система или её часть под определённой нагрузкой. Также может служить для проверки и подтверждения других атрибутов качества системы, таких как масштабируемость, надёжность и потребление ресурсов.

Существует особый подвид таких тестов, когда делается попытка достижения количественных пределов, обусловленных характеристиками самой системы и ее операционного окружения:

– продемонстрировать, что система удовлетворяет критериям производительности при заданных условиях;

– измерить, какая часть системы является причиной «плохой» производительности системы;

– измерить время реакции на действие пользователя, время отклика на запрос, и т.д.

При анализе требований основной упор необходимо сделать на определение основных критериев успешности проведенных тестов. Для этого необходимо будет выделить следующие характеристики:

– время отклика (время необходимое для открытия страницы или получения ожидаемого результата);

– интенсивность (число запросов в секунду);

– используемые ресурсы (загрузка процессора, количество используемой памяти и т.д.);

– максимальное количество пользователей (определяет число пользователей, способных работать с системой в условиях заданной конфигурации).

Заданные в требованиях характеристики, будут являться базовыми нагрузочными точками тестируемого приложения. Все получаемые результаты будут сравниваться с ними для принятия решения о завершении тестирования либо дальнейшем профилировании производительности.

Постепенное увеличение нагрузки, добавляя новых пользователей с некоторым интервалом времени, позволяет определить:

– измерение времени выполнения выбранных операций при определенных интенсивностях выполнения этих операций;

– количество пользователей, способных одновременно работать с приложением;

– границы приемлемой производительности при увеличении нагрузки;

– производительность при разных нагрузках.

Стрессовое тестирование (Stress testing) - обычно используется для понимания пределов пропускной способности приложения. Этот тип тестирования проводится для определения надёжности системы во время экстремальных или диспропорциональных нагрузок и отвечает на вопросы о достаточной производительности системы в случае, если текущая нагрузка сильно превысит ожидаемый максимум.

Увеличивая интенсивность операций выше пиковых (максимально разрешенных) значений, либо увеличивая количество пользователей до тех пор, пока нагрузка не станет выше максимально допустимых значений, проверяется, что система работоспособна в условиях стресса. Далее, опустив нагрузку до средних значений, проверяется (способность системы к регенерации), что система вернулась к нормальному состоянию (основные нагрузочные характеристики не превышают базовые).

Нагрузочное тестирование (Load testing) - это простейшая форма тестирования производительности. Нагрузочное тестирование обычно проводится для того, чтобы оценить поведение приложения под заданной ожидаемой нагрузкой. Этой нагрузкой может быть, например, ожидаемое количество одновременно работающих пользователей приложения, совершающих заданное число транзакций за интервал времени.

Такой тип тестирования обычно позволяет получить время отклика всех самых важных бизнес-транзакций. В случае наблюдения за базой данных, сервером приложений, сетью и т. д. этот тип тестирования может также идентифицировать некоторые узкие места приложения.

На результаты нагрузочного тестирования могут влиять разные факторы, такие как конфигурация тестового стенда, загруженность сети, заполненность базы данных и многие другие.

Нужно отметить те части конфигурации, которые требуют особого внимания:

Hardware:

– процессор (тип, частота, количество ядер и т.д);

– оперативная память (тип, объем, тайминг, эффективная частота и т.д.); – жесткие диски (тип, скорость и т.д.);

Software:

– операционная система;

– драйвера;

Network:

– топология сети;

– пропускная способность;

– протокол передачи данных;

Application:

– архитектура;

– база данных (структура + данные);

– программное обеспечение, необходимое для работы приложения (например, для Java приложений - JVM).

Основные причины, по которым не всегда получается продублировать конфигурацию системы на тестовом стенде:

1) сложность дублирования дорогого серверного железа для тестовых нужд;

2) ограничения на использование лицензий требуемого программного обеспечения;

3) закрытость архитектуры приложения со стороны заказчика по соображениям безопасности;

4) трудность воссоздания или транспортировки базы данных приложения;

5) сложность воссоздания требуемой архитектуры сети.

Разработка модели нагрузки. Для этого необходимо определить следующее:

– список тестируемых операций;

– интенсивность выполнения операций;

– зависимость изменения интенсивности выполнения операций от времени.

В список тестируемых задач должны войти операции, критичные с точки зрения бизнеса, а также с технической точки зрения:

– критичными с точки зрения бизнеса являются операции, скорость выполнения которых, реально влияет на производительность бизнес- процесса. Например, увеличение длительности обслуживания клиентов в банке, невозможность выполнения необходимого количества операций в течение дня и так далее;

– критичными с технической точки зрения являются ресурсоемкие операции, требующие большое количество памяти, серьезно задействующие процессор, создающие значительный сетевой трафик. Как правило, это операции, выполняемые одновременно большим количеством бизнес- пользователей или создание сложных отчетов, в которые входят так называемые "тяжелые" запросы к базе данных.

Тестирование стабильности (Stability testing) проводится с целью убедиться в том, что приложение выдерживает ожидаемую нагрузку в течение длительного времени. При проведении этого вида тестирования осуществляется наблюдение за потреблением приложением памяти, чтобы выявить потенциальные утечки. Такое тестирование выявляет деградацию производительности, выражающуюся в снижении скорости обработки информации и/или увеличении времени ответа приложения после продолжительной работы по сравнению с началом теста.

Используя базовый нагрузочный профильзапускается тест длительностью от нескольких часов до нескольких дней с целью выявления утечек памяти, перезапуска серверов и других аспектов влияющих на нагрузку.

Тестирование удобства использования (Usability testing) - исследование, выполняемое с целью определения, удобен ли некоторый искусственный объект (такой как веб-страница, пользовательский интерфейс или устройство) для его предполагаемого применения. Таким образом, проверка эргономичности измеряет эргономичность объекта или системы.

Проверка эргономичности сосредоточена на определённом объекте или небольшом наборе объектов, в то время как исследования взаимодействия человек-компьютер в целом — формулируют универсальные принципы. Это метод оценки удобства продукта в использовании, основанный на привлечении пользователей в качестве тестировщиков, испытателей и суммировании полученных от них выводов.

При испытании многих продуктов пользователю предлагают в «лабораторных» условиях решить основные задачи, для выполнения которых этот продукт разрабатывался и просят высказывать во время выполнения этих тестов свои замечания. Процесс тестирования фиксируется в протоколе (логе) и/или на аудио- и видеоустройства — с целью последующего более детального анализа.

Если юзабилити-тестирование выявляет какие-либо трудности (например сложности в понимании инструкций, выполнении действий или интерпретации ответов системы), то разработчики должны доработать продукт и повторить тестирование. Наблюдение за тем, как люди взаимодействуют с продуктом, нередко позволяет найти для него более оптимальные решения.

Если при тестировании используется модератор, то его задача — держать респондента сфокусированным на задачах (но при этом не "помогать" ему решать эти задачи).

Основную трудность после проведения процедуры юзабилити-тестирования нередко представляют большие объёмы и беспорядочность полученных данных. Поэтому для последующего анализа важно зафиксировать:

1) речь модератора и респондента;

2) выражение лица респондента (снимается на видеокамеру);

3) изображение экрана компьютера, с которым работает респондент;

4) различные события, происходящие на компьютере, связанные с действиями пользователя:

5) перемещение курсора и нажатия на клавиши мыши;

6) использование клавиатуры;

7) переходы между экранами (браузера или другой программы).

Все эти потоки данных должны быть синхронизированы по тайм-кодам, чтобы при анализе их можно было бы соотносить между собой.

Наряду с модератором в тестировании нередко участвуют наблюдатели. По мере обнаружения проблем они делают свои заметки о ходе тестирования так, чтобы после можно было синхронизировать их с основной записью. В итоге каждый значимый фрагмент записи теста оказывается прокомментирован в заметках наблюдателя.

В идеале ведущий (т.е. модератор) представляет разработчика, наблюдатели — заказчика, а испытатели — конечного пользователя (например, покупателя).

Существует еще один подход к usability-тестированию: для решения задачи, предложенной пользователю, разрабатывается "идеальный" сценарий решения этой задачи. Как правило, это сценарий, на который ориентировался разработчик. При выполнении задачи пользователями регистрируются их отклонения от задуманного сценария для последующего анализа. После нескольких итераций доработки программы и последующего тестирования можно получить удовлетворительный интерфейс с точки зрения пользователя.

Тестирование интерфейса пользователя (UI testing) - тестирование графического интерфейса пользователя для того, чтобы убедиться, что он соответствует принятым стандартам и их требованиям. Проверяется, как приложение обрабатывает ввод с клавиатуры и «мышки» и как отображаются элементы графического интерфейса (текст, кнопки, меню, списки и прочие элементы).

Тестирование безопасности (security testing) - оценка уязвимости программного обеспечения к различным атакам. Компьютерные системы очень часто являются мишенью незаконного проникновения. Под проникновением понимается широкий диапазон действий: попытки хакеров проникнуть в систему из спортивного интереса, месть рассерженных служащих, взлом мошенниками для незаконной наживы.

Тестирование безопасности проверяет фактическую реакцию защитных механизмов, встроенных в систему, на проникновение. В ходе тестирования безопасности испытатель играет роль взломщика. Ему разрешено все:

– попытки узнать пароль с помощью внешних средств;

– атака системы с помощью специальных утилит, анализирующих защиты;

– подавление, ошеломление системы (в надежде, что она откажется обслуживать других клиентов);

– целенаправленное введение ошибок в надежде проникнуть в систему в ходе восстановления;

– просмотр несекретных данных в надежде найти ключ для входа в систему.

Тестирование локализации (Localization testing) - многогранная вещь, подразумевающая проверку множества аспектов, связанных с адаптацией продукта для пользователей из других стран.

Тестирование совместимости (Compatibility testing) — вид нефункционального тестирования, основной целью которого является проверка корректной работы продукта в определенном окружении. Окружение может включать в себя следующие элементы:

– аппаратная платформа;

– сетевые устройства;

– периферия;

– операционная система;

– СУБД;

– системное программное обеспечение - утилиты;

– браузеры.

Автоматизированное тестирование ПО (Automation Testing) - это процесс верификации программного обеспечения, при котором основные функции и шаги теста такие как запуск, инициализация, выполнение, анализ и выдача результата, выполняются автоматически при помощи инструментов для автоматизированного тестирования.

Преимущества автоматизации тестирования:

– повторяемость – все написанные тесты всегда будут выполняться однообразно, то есть исключен «человеческий фактор». Тестировщик не пропустит тест по неосторожности и ничего не напутает в результатах;

– быстрое выполнение – автоматизированному скрипту не нужно сверяться с инструкциями и документациями, это сильно экономит время выполнения;

– меньшие затраты на поддержку – когда автоматические скрипты уже написаны, на их поддержку и анализ результатов требуется, как правило, меньшее время чем на проведение того же объема тестирования вручную;

– отчеты – автоматически рассылаемые и сохраняемые отчеты о результатах тестирования;

– выполнение без вмешательства – во время выполнения тестов инженер-тестировщик может заниматься другими полезными делами, или тесты могут выполняться в нерабочее время (этот метод предпочтительнее, так как нагрузка на локальные сети ночью снижена).

Недостатки автоматизации тестирования (их тоже немало):

– повторяемость – все написанные тесты всегда будут выполняться однообразно. Это одновременно является и недостатком, так как тестировщик, выполняя тест вручную,

может обратить внимание на некоторые детали и, проведя несколько дополнительных операций, найти дефект. Скрипт этого сделать не может;

– затраты на поддержку – несмотря на то, что в случае автоматизированных тестов они меньше, чем затраты на ручное тестирование того же функционала – они все же есть. Чем чаще изменяется приложение, тем они выше;

– большие затраты на разработку – разработка автоматизированных тестов это сложный процесс, так как фактически идет разработка приложения, которое тестирует другое приложение. В сложных автоматизированных тестах также есть фреймворки, утилиты, библиотеки и прочее. Естественно, все это нужно тестировать и отлаживать, а это требует времени;

– стоимость инструмента для автоматизации – в случае если используется лицензионное ПО, его стоимость может быть достаточно высока. Свободно распространяемые инструменты как правило отличаются более скромным функционалом и меньшим удобством работы;

– пропуск мелких ошибок - автоматический скрипт может пропускать мелкие ошибки, на проверку которых он не запрограммирован. Это могут быть неточности в позиционировании окон, ошибки в надписях, которые не проверяются, ошибки форм, с которыми не осуществляется взаимодействие во время выполнения скрипта.

Автоматизацию нужно применять в следующих случаях:

1) труднодоступные места в системе (бэкенд процессы, логирование файлов, запись в БД);

2) часто используемая функциональность, риски от ошибок в которой достаточно высоки. Автоматизировав проверку критической функциональности, можно гарантировать быстрое нахождение ошибок, а значит и быстрое их решение;

3) рутинные операции, такие как переборы данных (формы с большим количеством вводимых полей. Автоматизировать заполнение полей различными данными и их проверку после сохранения);

4) валидационные сообщения (автоматизировать заполнение полей не корректными данными и проверку на появление той или иной валидации);

5) длинные end-to-end сценарии;

6) проверка данных, требующих точных математических расчетов;

7) проверка правильности поиска данных.

Для более эффективного использования автоматизации тестирования лучше разработать отдельные тест-кейсы проверяющие:

– базовые операции создания/чтения/изменения/удаления сущностей (так называемые CRUD операции - Create / Read / Update / Delete). Пример: создание, удаление, просмотр и изменение данных о пользователе;

– типовые сценарии использования приложения, либо отдельные действия. Пример: пользователь заходит на почтовый сайт, листает письма, просматривает новые, пишет и отправляет письмо, выходит с сайта. Это end-to-end сценарий, который проверяет совокупность действий;

– интерфейсы, работы с файлами и другие моменты, неудобные для тестирования вручную. Пример: система создает некоторый xml-файл, структуру которого необходимо проверить. Это и есть функциональность, от автоматизации тестирования которой, можно получить наибольшую отдачу.

Основные функции скрипта:

1) precondition:

– инициализация приложения (например, открытие главной страницы, вход под тестовым пользователем, переход в необходимую часть приложения и подведение системы к состоянию пригодному для тестирования);

– инициализация тестовых данных;

2) steps:

– непосредственное проведение теста;

– занесение данных о результате теста, с обязательным сохранением причин провала и шагов, по которым проходил тест;

3) post condition:

– удаление, созданных в процессе выполнения скрипта, ненужных тестовых данных;

– корректное завершение работы приложения